Principles of Distributed Systems

Week 2: Distributed System Architectures [第2周：分布式系统架构]

By: Fahad Sabah

计算机学院,
北京理工大学,中国
College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn

Recap of Week1 第一周回顾

1. What is a distributed system?
2. Why do we build it?
3. What are the Core Principles?
4. What major types exist?
5. Project Discussion

1. 什么是分布式系统?
2. 我们为什么要建造它?
3. 核心原则是什么?
4. 有哪些主要类型存在?
5. 项目讨论

Contents 目录

1. Architectural Styles

2. System Architectures

3. Middleware

4. Self-Management

5. Practical: simple client-server architecture and socket programming

1. 建筑风格

2. 系统架构

3. 中间件

4. 自我管理

5. 实用：简单的客户端-服务器架构和套接字编程

建筑风格与集中式建筑
Architectural Styles and Centralized Architectures

Learning objectives

- ❑ explain what architecture means in distributed systems
- ❑ identify components and connectors
- ❑ describe common architectural styles
- ❑ explain what a centralized architecture is
- ❑ give simple real-world examples of centralized systems
- ❑ discuss the main advantages and disadvantages of centralized architecture

学习目标

- ❑ 解释分布式系统中的架构含义
- ❑ 识别组件和连接器
- ❑ 描述常见的建筑风格
- ❑ 解释什么是集中式架构
- ❑ 给出一些简单的现实世界中集中式系统的例子
- ❑ 讨论集中式架构的主要优缺点

热身
Warm-up

When we use WeChat, Taobao, or Baidu, is there only one server behind it?

In real systems, there are usually many parts working together.

To understand such systems, we need to study their architecture.

Architecture = the structure of the system.

我们用微信、淘宝或百度时，背后只有一个服务器吗？

在真实系统中，通常有许多部分协同工作。

要理解这些系统，我们需要研究它们的架构。

架构=系统的结构。

什么是建筑?
What is architecture?

Architecture means:

- ❑ what parts the system has
- ❑ what each part does
- ❑ how the parts communicate
- ❑ how the whole system is organized

建筑学意味着：

- ❑ 系统包含哪些部分
- ❑ 每个部件的作用
- ❑ 各部分的交流方式
- ❑ 整个系统的组织方式

组成部分
Components

Components are the main building blocks of the system.

Examples of components

- ❑ Clients
- ❑ Servers
- ❑ Databases
- ❑ Services
- ❑ Caches
- ❑ message queues

组件是系统的主要构建单元。

组件示例

- ❑ 客户
- ❑ 服务器
- ❑ 数据库
- ❑ 服务
- ❑ 缓存
- ❑ 消息队列

连接器
Connectors

Connectors are the paths or mechanisms that let components communicate.

Examples

- network links
- APIs
- Protocols
- message passing
- RPC calls

连接器是让组件之间通信的路径或机制。

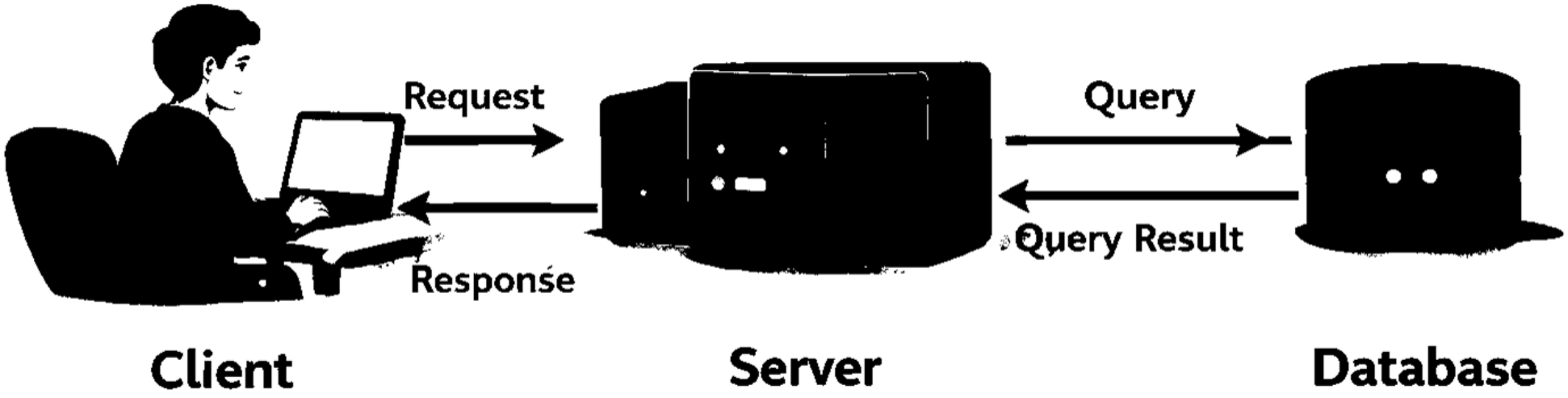
示例

- 网络连接
- API
- 协议
- 消息传递
- RPC 调用

快速示意
Quick diagram

Client → Server → Database

客户端→服务器→数据库



建筑风格
Architectural styles

What is an architectural style?

An architectural style is a common pattern for organizing a system.

什么是建筑风格?

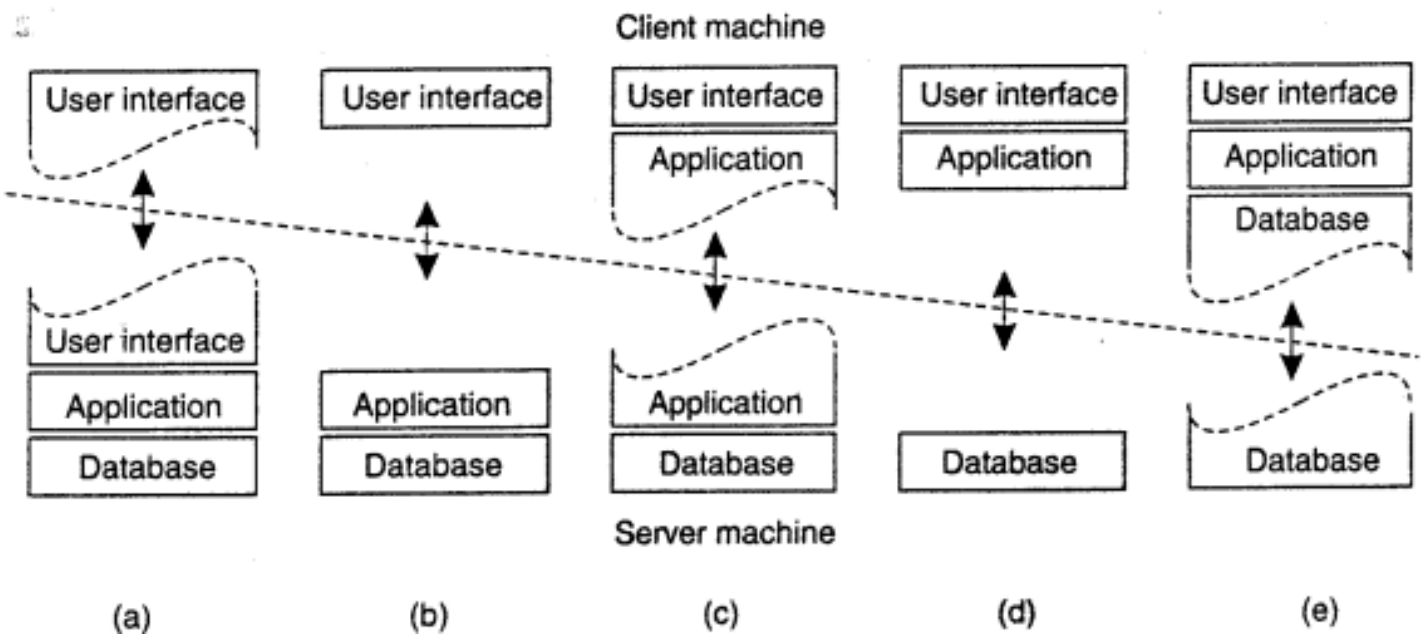
建筑风格是组织系统时常见的模式。

简洁的建筑风格
Simple architectural styles

1. **Client–Server**
   - ❑ clients ask for services
   - ❑ servers provide services
   - ❑ Examples:
     - browser and web server
     - app and login server

1. 客户端-服务器

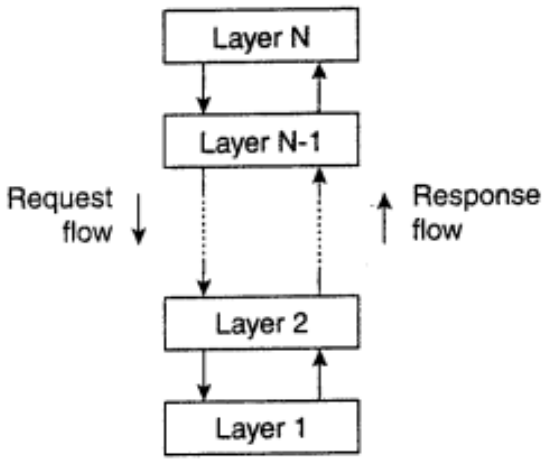
- ❑ 客户需要服务
- ❑ 服务器提供服务
- ❑ 示例：
  - 浏览器和网页服务器
  - 应用和登录服务器



简洁的建筑风格
Simple architectural styles

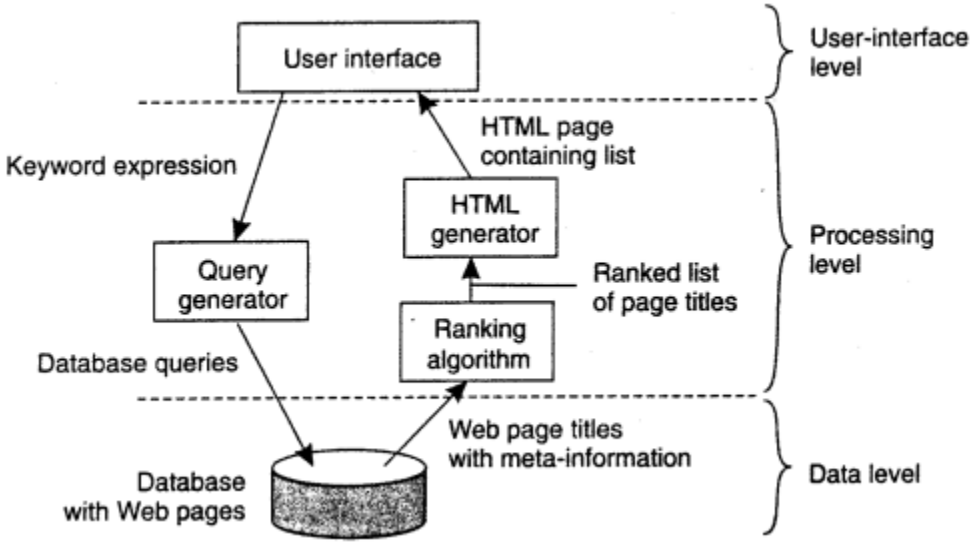
2. Layered systems

- system is divided into layers
- each layer has a specific role
- Examples:
  - presentation layer
  - application layer
  - data layer



2. 分层系统

- 系统被划分为多个层
- 每一层都有特定的角色
- 示例：
  - 展示层
  - 应用层
  - 数据层



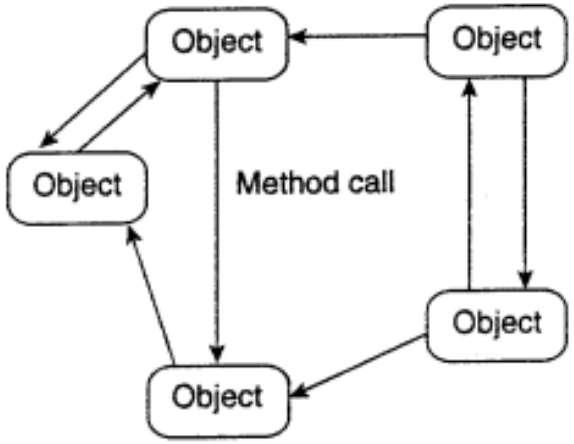
简洁的建筑风格
Simple architectural styles

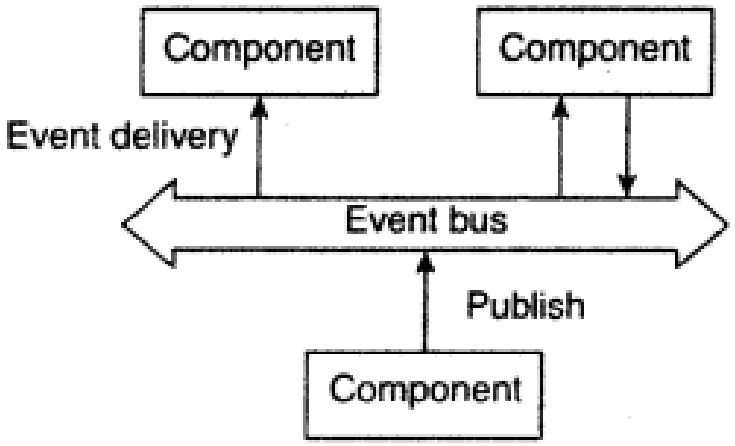
3. Object/service-based systems

- ❑ system is made of services or objects
- ❑ each one provides a specific function
- ❑ Example:
  - payment service
  - search service
  - notification service

3. 基于对象/服务的系统

- ❑ 系统由服务或对象组成
- ❑ 每个人都有特定的功能
- ❑ 示例：
  - 支付服务
  - 检索服务
  - 通知服务





系统架构
System architectures

1. Centralized architecture
2. De-centralized architecture
3. Hybrid architecture

1. 集中式架构
2. 集中式架构
3. 混合架构

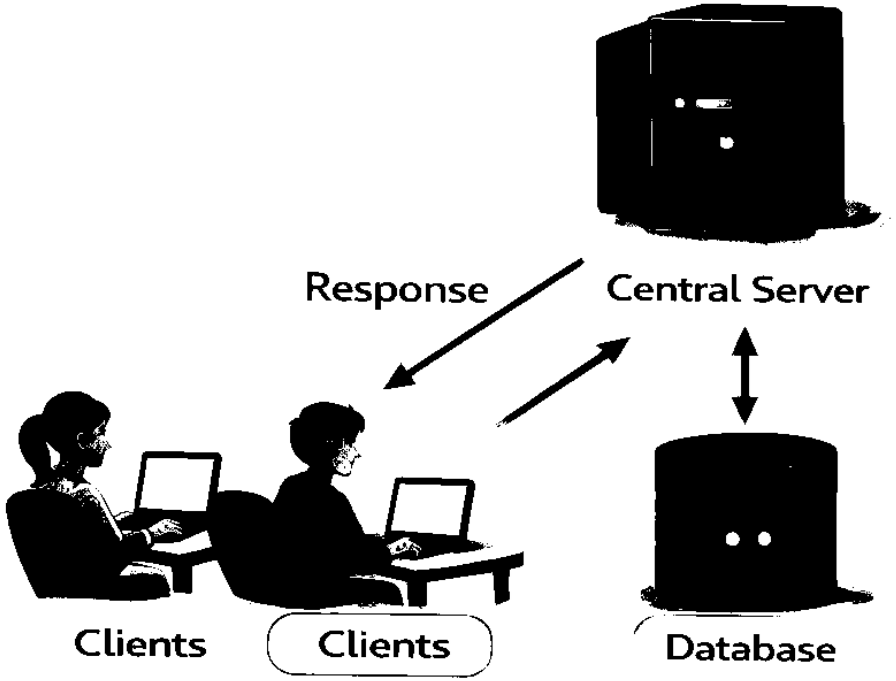
集中式架构
Centralized architecture

主要思想

- ❑ 一台服务器，或少数中央服务器提供服务
- ❑ 客户端向该中心点发送请求

Main idea

- ❑ one server, or a small number of central servers, provides the service
- ❑ clients send requests to this central point



集中式架构（续）
Centralized architecture (contd.)

Basic structure

- ❑ Many clients → one main server → one database
- ❑ Explanation
- ❑ users connect to one main place
- ❑ decisions and services are mostly controlled centrally
- ❑ data may also be stored centrally

How it works

- ❑ client sends request
- ❑ central server receives request
- ❑ server processes the request
- ❑ server may read/write data from database
- ❑ server sends response back

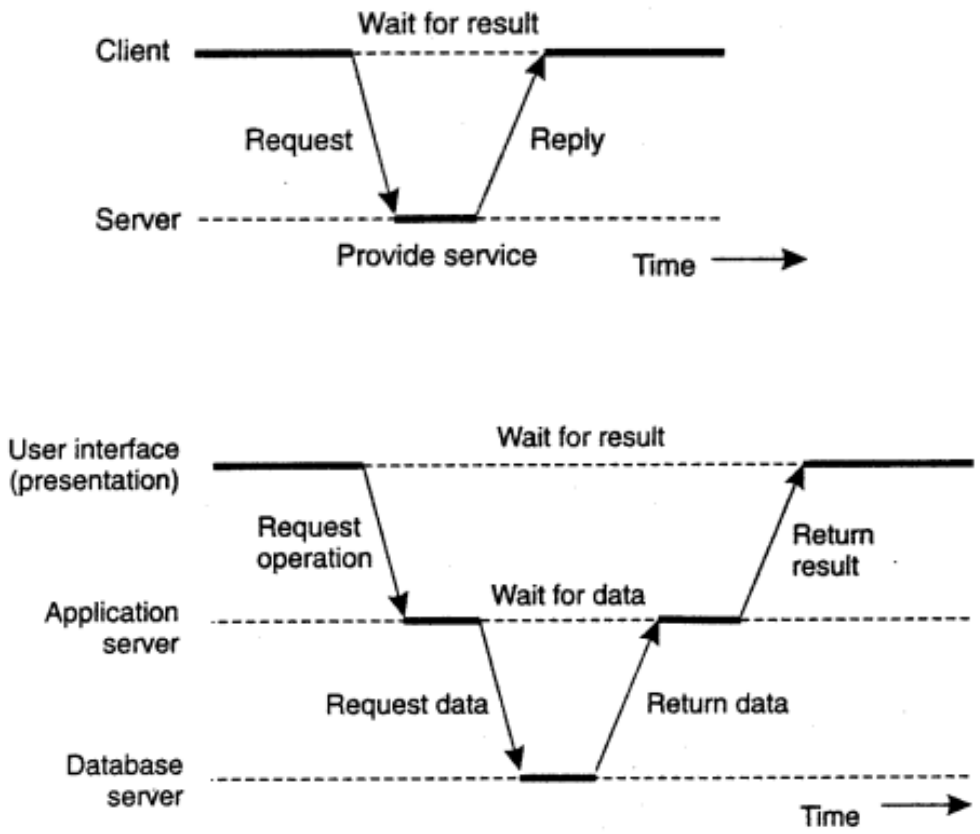


Figure 2-6. An example of a server acting as client.

集中式建筑的例子
Examples of centralized architecture

Example 1: University website

- ❑ students open the site
- ❑ requests go to a university server
- ❑ server returns pages or results

❑ **Why centralized?**

Because one main server or server cluster handles requests from all users.

示例1：大学官网

学生们开放网站
请求会发送到大学服务器
服务器返回页面或结果

为什么要集中？

因为一个主服务器或服务器集群负责处理所有用户的请求。

集中式建筑的例子
Examples of centralized architecture

Example 2: Single database-backed campus portal

- ❑ login page
- ❑ class schedule
- ❑ Grades
- ❑ Notices
- ❑ All connected to one central backend.

❑ **Why centralized?**

Because the services are controlled from one main center.

示例2：单一数据库支持的校园门户

登录页面

课程安排

等级

公告

所有设备都连接到一个中央后端。

为什么要集中？

因为服务由一个主要中心控制。

集中式建筑的例子
Examples of centralized architecture

Example 3: Simple online library system

- ❑ student searches a book
- ❑ central server checks central database
- ❑ result is returned to student

❑ **Why centralized?**

Because book records and search logic are in one central system.

示例3：简单的在线图书馆系统

- ❑ 学生搜索一本书
- ❑ 中央服务器检查中央数据库
- ❑ 结果返回给学生

为什么要集中？

因为书籍记录和检索逻辑集中在一个中央系统中。

集中式建筑的例子
Examples of centralized architecture

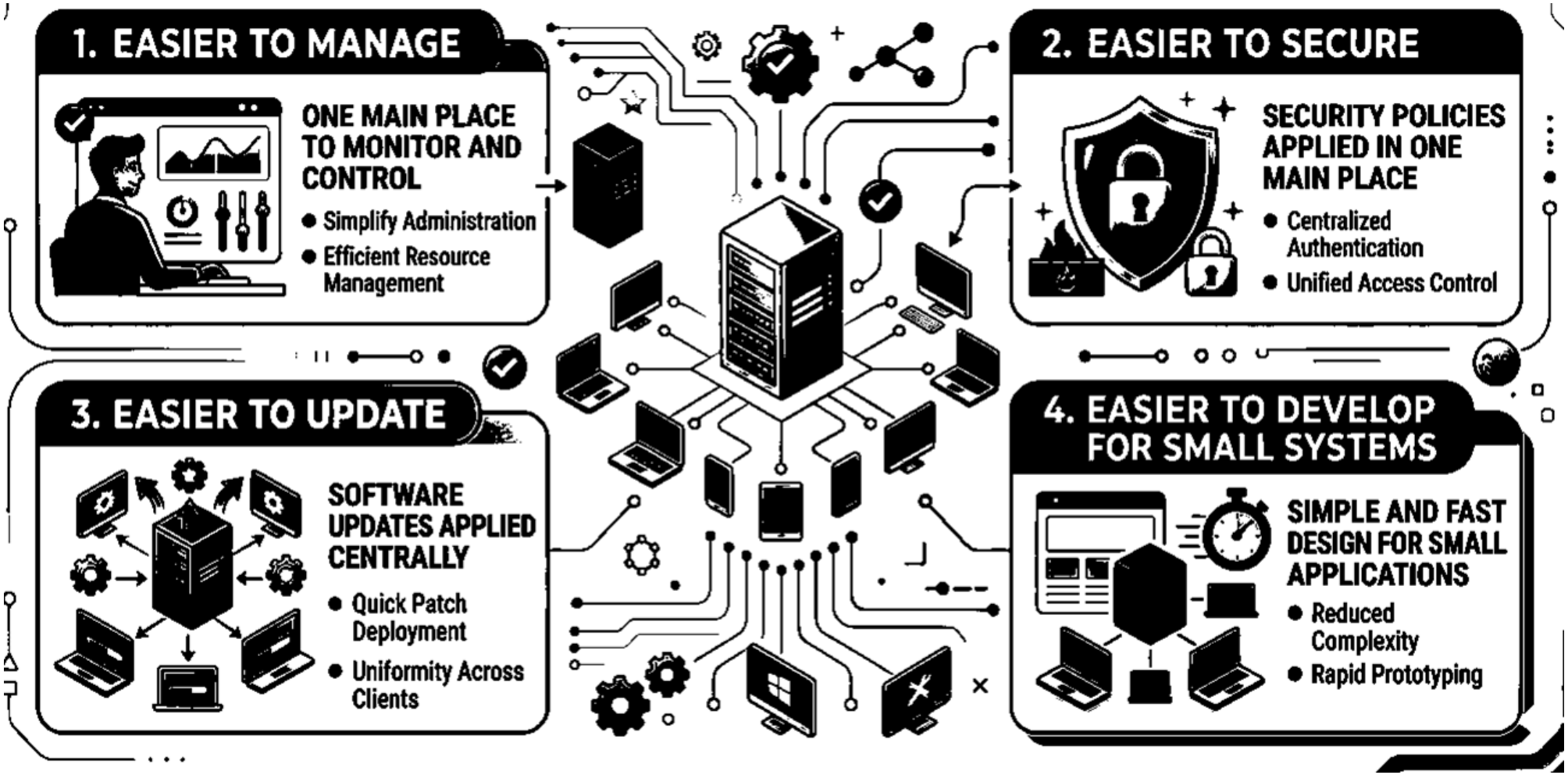
Example 4: Attendance checking system

- ❑ teacher submits attendance
- ❑ all attendance records go to one central database
- ❑ students check attendance from same system

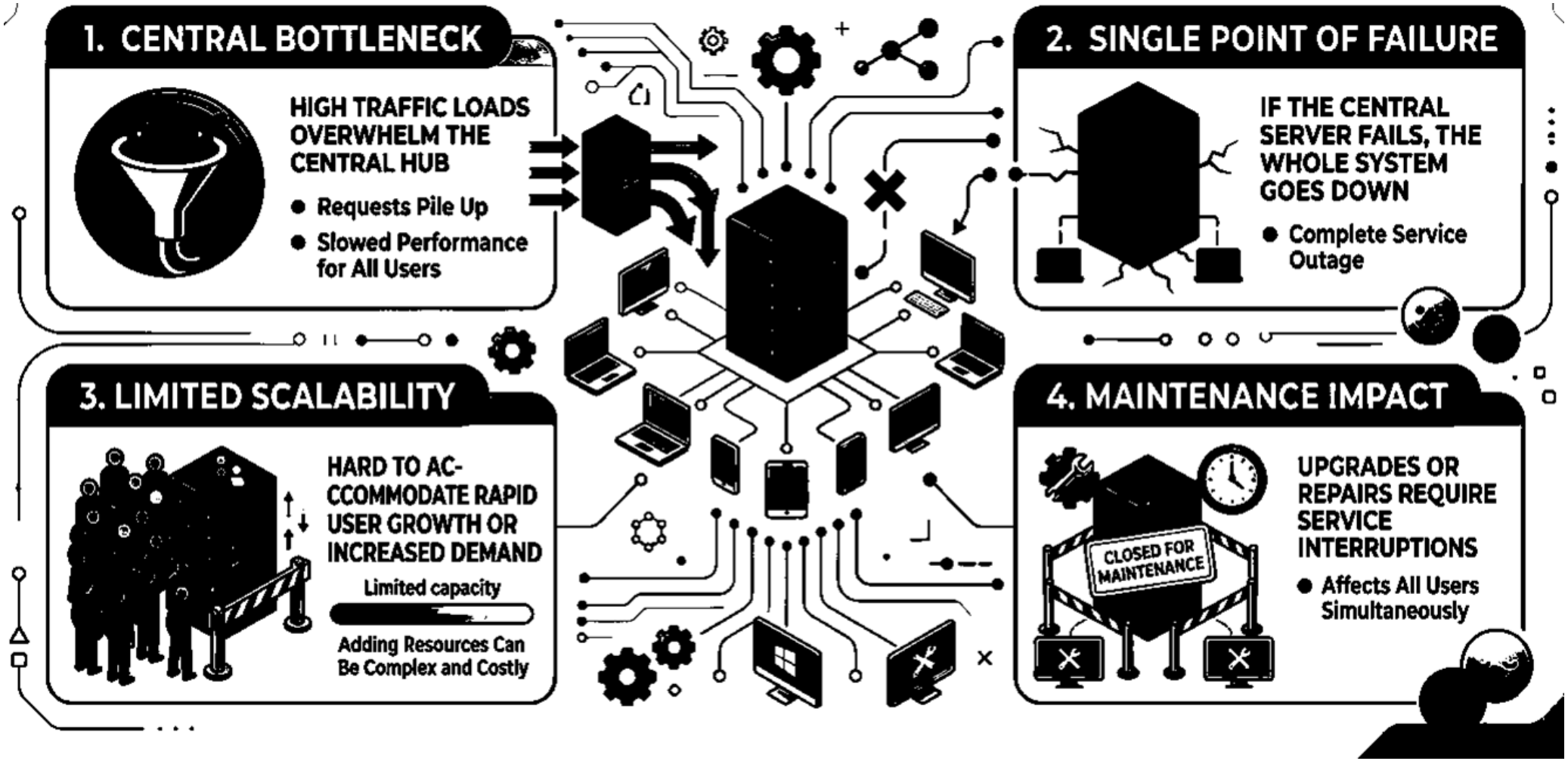
示例4：出勤检查系统

- ❑ 教师提交出勤情况
- ❑ 所有出勤记录都汇入一个中央数据库
- ❑ 学生们通过同一系统核查出勤情况

集中式架构的优势
Advantages of centralized architecture



集中式架构的缺点
Disadvantages of centralized architecture



迷你活动
Mini activity

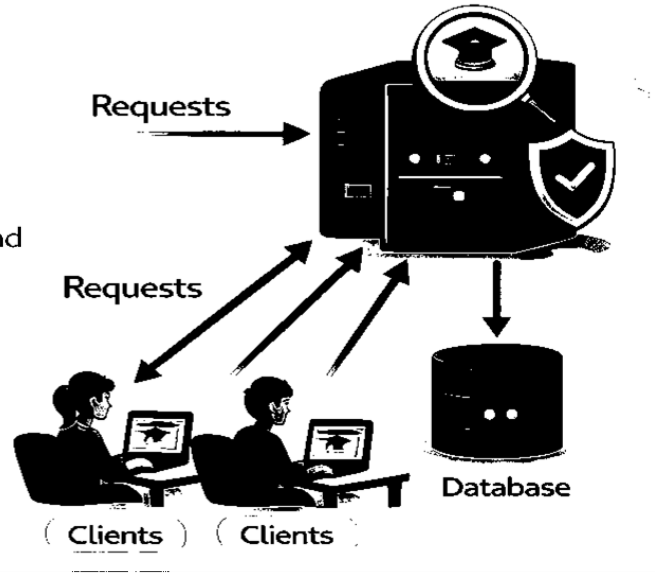
Is a campus result-checking portal usually centralized or decentralized?

Answer: Centralized!

- one university controls it
- one official database stores results
- students request data from one central backend

Easier to develop for small systems

For small applications, centralized design is often simpler and faster.



Why is centralized good here?

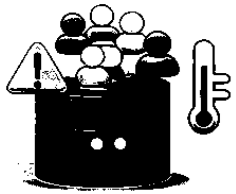
Benefits:

- easy to manage
- trusted source
- single reliable database

Problems during result release day?

Problems:

- may become slow
- could crash
- high server load



简单的现实世界类
Simple real-world analogy

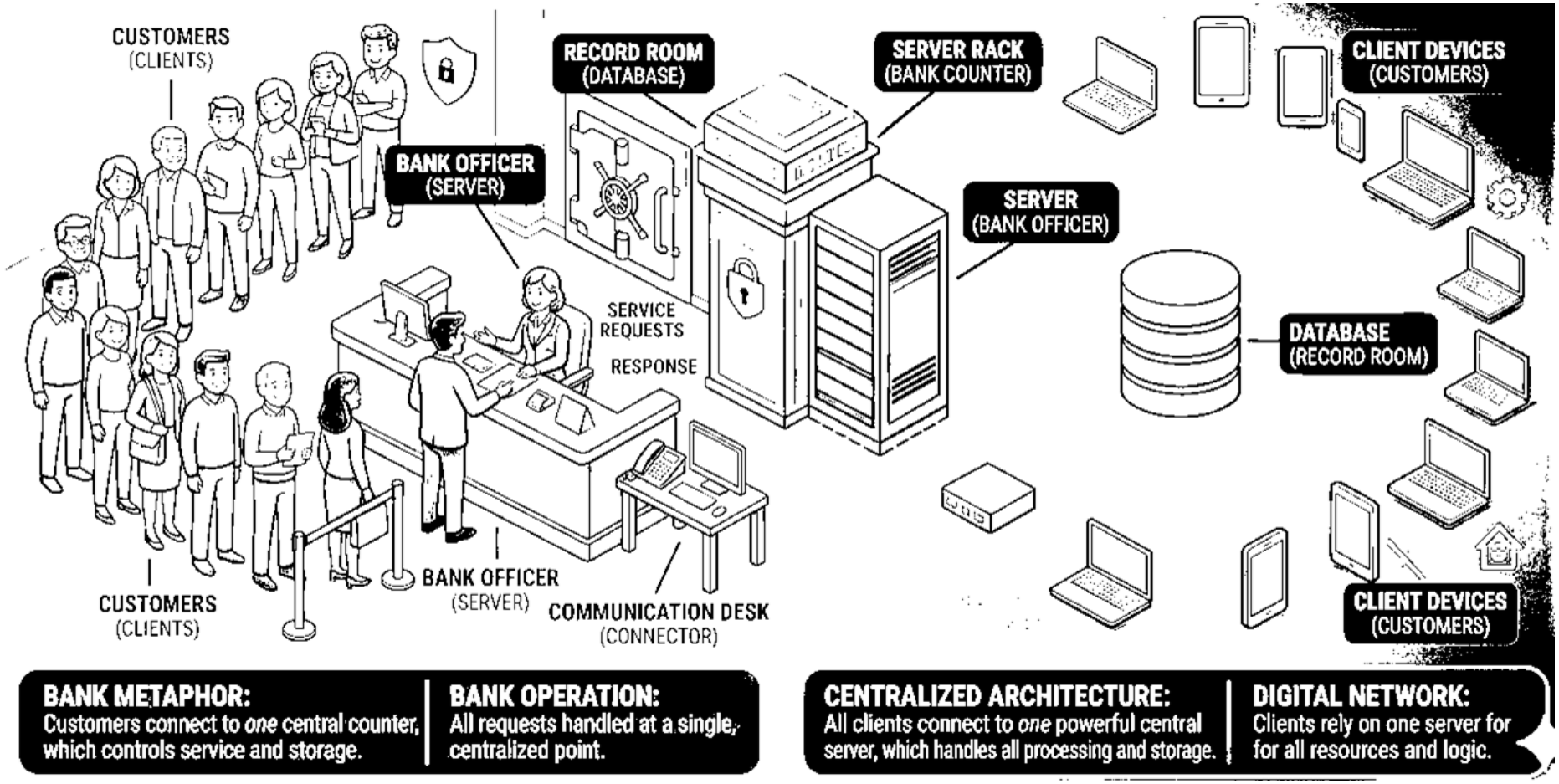
Centralized architecture = bank counter

- ❑ customers go to one counter
- ❑ the counter controls service
- ❑ records are stored centrally

Components

- ❑ customer = client
- ❑ bank officer = server
- ❑ record room = database
- ❑ communication desk = connector

简单的现实世界类
Simple real-world analogy



摘要
Summary

Key points to remember

Architecture

- ❑ The overall structure of a distributed system.

Components

- ❑ The parts that do the work.

Connectors

- ❑ The communication paths between parts.

Architectural style

- ❑ A common pattern for organizing the system.

Centralized architecture

- ❑ A design in which one central server or a small set of central servers provides services to clients.

需要记住的关键点

建筑

- ❑ 分布式系统的整体结构。

组成部分

- ❑ 那些做工作的部分。

连接器

- ❑ 各部分之间的通信路径。

建筑风格

- ❑ 这是组织系统的常见模式。

集中式架构

- ❑ 一种设计中，一台中央服务器或一小部分中央服务器为客户端提供服务。

快速回顾
Quick recap

- We learned that architecture means how a distributed system is organized.
- We identified components and connectors.
- We looked at a few architectural styles.
- Then we focused on centralized architecture, which is simple and easy to manage, but may suffer from bottlenecks, single-point failure, and limited scalability.

- 我们了解到，架构指的是分布式系统的组织方式。
- 我们确定了组件和连接器。
- 我们考察了几种建筑风格。
- 随后我们专注于集中式架构，这种架构简单易管理，但可能存在瓶颈、单点故障和有限的可扩展性。

Quiz

Optional homework

Draw a simple centralized system

- ❑ campus library
- ❑ campus portal
- ❑ online attendance system
- ❑ Include: clients server data

Distributed System Architecture Q1

Scan with WeChat or long-press

业

个简单的集中系统:

校园图书馆

校园门户

在线出勤系统

包括：客户端服务器数据库

Write a short paragraph: Why is a campus result system usually centralized?

写一段简短的话：为什么校园成绩系统通常是集中的？

去中心化与混合架构
Decentralized and Hybrid Architectures

去中心化与混合架构
Decentralized and Hybrid Architectures

Learning objectives

- ❑ explain what a decentralized architecture is
- ❑ explain what a hybrid architecture is
- ❑ compare centralized, decentralized, and hybrid systems
- ❑ identify the main advantages and disadvantages of each
- ❑ classify simple real-world examples
- ❑ understand why many real systems are hybrid

学习目标

- ❑ 解释什么是去中心化架构
- ❑ 解释什么是混合架构
- ❑ 比较集中式、去中心化和混合式系统
- ❑ 明确每种方式的主要优缺点
- ❑ 分类简单的现实世界例子
- ❑ 理解为什么许多真实系统是混合的

去中心化架构
Decentralized architecture

简单定义

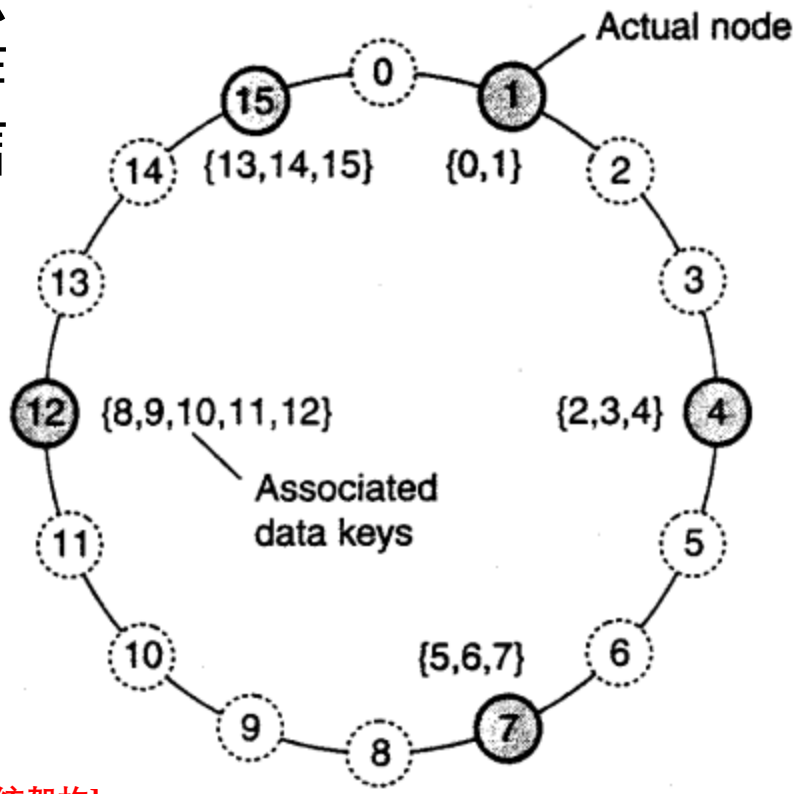
去中心化架构是一种系统，满足以下条件：

- ❑没有单一的中央控制点
- ❑许多节点可以独立工作
- ❑节点之间可以直接通信

Simple definition

A decentralized architecture is a system where:

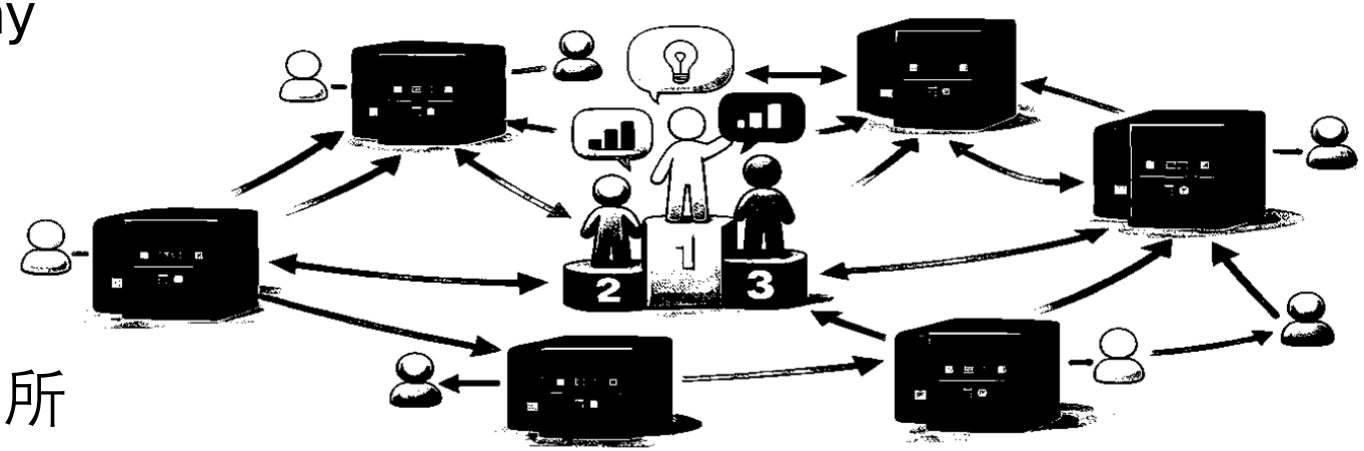
- ❑ there is no single central control point
- ❑ many nodes can work independently
- ❑ nodes may communicate directly with each other



去中心化架构
Decentralized architecture

In a decentralized system:
control is spread outwork is spread out
decisions may be shared among many nodes

Decentralized = no single boss for everything.
去中心化 = 没有一个单独的中心控制所有事情。



去中心化架构
Decentralized architecture

Simple analogy

Centralized
A classroom where every student must ask one teacher for every action.

Decentralized
A group discussion where students can talk directly to each other and cooperate without one central controller for every step.

简单的类比

集中式
一个每个学生都必须向一位老师请求每个动作的教室。

去中心化
这是一个小组讨论，学生可以直接交流并协作，无需每一步都有一个中央控制者。

去中心化架构
Decentralized architecture

Basic structure

Node A ↔ Node B ↔ Node C ↔ Node D

Explanation:

- ❑ each node can communicate
- ❑ there may be no permanent main server
- ❑ nodes may act as both client and server

基本结构

节点A ↔ 节点 B ↔ 节点 C ↔ 节点 D 节点

解释：

- ❑ 每个节点都可以通信
- ❑ 可能没有永久的主服务器
- ❑ 节点可以同时作为客户端和服务器

去中心化架构的示例
Examples of decentralized architecture

示例1：点对点文件共享

- ❑ 用户直接分享文件
- ❑ 一个用户可以从多个节点下载
- ❑ 同龄人可以加入或离开

为什么选择去中心化?

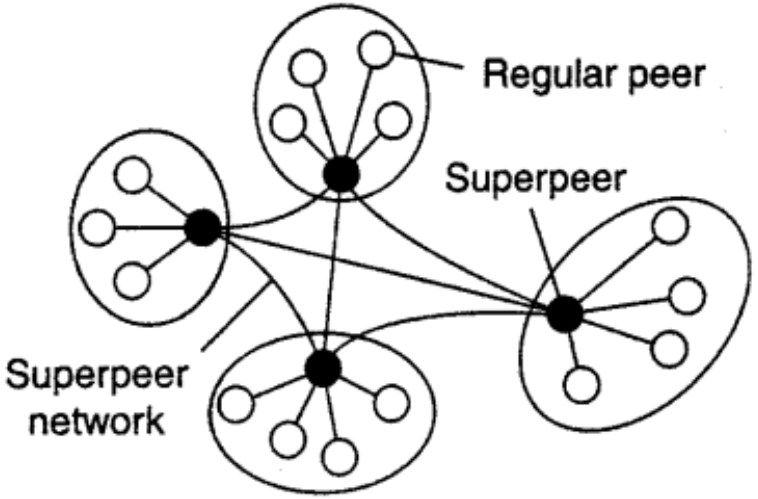
因为数据和服务是由多个节点提供，而不是一个主服务器。

Example 1: Peer-to-peer file sharing

- ❑ users share files directly
- ❑ one user can download from many peers
- ❑ peers may join and leave

Why decentralized?

Because data and service are provided by many peers, not one main server.



去中心化架构的示例
Examples of decentralized architecture

Example 2: Blockchain / distributed ledger

- ❑ copies of data exist on many nodes
- ❑ nodes cooperate to validate updates
- ❑ no single machine owns the whole truth alone

Why decentralized?

Because control and record-keeping are shared across nodes.

示例2：区块链/分布式账本

- ❑ 数据副本存在于许多节点上
- ❑ 节点协作验证更新
- ❑ 没有哪台机器能独自拥有全部真相

为什么选择去中心化?

因为控制和记录管理在节点间共享。

去中心化架构的示例
Examples of decentralized architecture

Example 3: Sensor networks

- many sensors collect data
- different nodes may process or forward information
- system does not always depend on one central node for every step

示例3：传感器网络

- 许多传感器收集数据
- 不同节点可能处理或转发信息
- 系统并非每一步都依赖于一个中心节点

Example 4: Ad hoc mobile networks

- devices communicate directly
- topology can change dynamically
- nodes can relay information for each other

示例4：临时移动网络

- 设备直接通信
- 拓扑结构可以动态变化
- 节点之间可以传递信息

去中心化架构的优势
Advantages of decentralized architecture

1. **No single point of failure**
If one node fails, the whole system may still continue.
2. Better scalability in some cases
As more nodes join, the system may gain more resources.
3. Resource sharing
Different nodes can contribute:
storage
Computing
bandwidth
4. Better resilience
Some decentralized systems can continue even when parts fail or leave.

1. 没有单点故障
如果一个节点故障，整个系统仍可继续运行。
2. 在某些情况下更具可扩展性
随着更多节点加入，系统可能获得更多资源。
3. 资源共享
不同节点可以贡献：
存储
计算
带宽
4. 更高的韧性
一些去中心化系统即使部分故障或离开，仍能继续运行。

去中心化架构的缺点
Disadvantages of decentralized architecture

1. Harder to coordinate
Without a center, it is more difficult to keep all nodes consistent and organized.

2. Harder to manage
Monitoring, debugging, and updating become more complex.

3. Harder to secure
Trust and access control are usually more difficult.

4. Data consistency can be difficult
Different nodes may have different views of the data.

1. 协调起来更难
没有中心，保持所有节点的一致性和组织更加困难。

2. 管理更难
监控、调试和更新变得更加复杂。

3. 更难获得
信任和访问控制通常更为复杂。

4. 数据一致性可能很困难
不同节点可能对数据有不同的视角。

混合架构
Hybrid architecture

A hybrid architecture combines features of:

- ❑ centralized systems
- ❑ decentralized systems

Hybrid = part central, part distributed.
混合架构 = 一部分集中，一部分分布。

混合架构结合了以下特征：

集中式系统
去中心化系统

混合型=部分中央，部分分布。
混合架构 = 一部分集中，一部分分布。

混合架构
Hybrid architecture

Why hybrid systems exist

- ❑ Pure centralized systems are simple, but they can become bottlenecks.
- ❑ Pure decentralized systems are flexible, but they can be hard to control.
- ❑ So many real systems choose a balance:
  - ❑ keep some central coordination
  - ❑ distribute some work across many nodes

混合系统存在的原因

- ❑ 纯集中式系统虽然简单，但也可能成为瓶颈。
- ❑ 纯去中心化系统具有灵活性，但控制起来可能较为困难。
- ❑ 许多真实系统选择了一种平衡：
  - ❑ 保持一些中央协调
  - ❑ 将部分工作分配到多个节点

混合架构
Hybrid architecture

Easy analogy

A school may have one central administration office, but students and teachers also communicate directly in classes, groups, and labs.

That is a hybrid model:

- ❑ some things are central
- ❑ some things are distributed

简单的比喻

学校可能有一个中央行政办公室，但学生和教师也直接在班级、小组和实验室中沟通。

这就是混合模式：

- ❑ 有些事情是核心的
- ❑ 有些东西是分布式的

混合架构
Hybrid architecture

Basic structure

Clients → central service

and also

nodes/services ↔ nodes/services

Explanation:

- ❑ login may be central
- ❑ caching may be distributed
- ❑ content delivery may be distributed
- ❑ coordination may still have a central part

基本结构

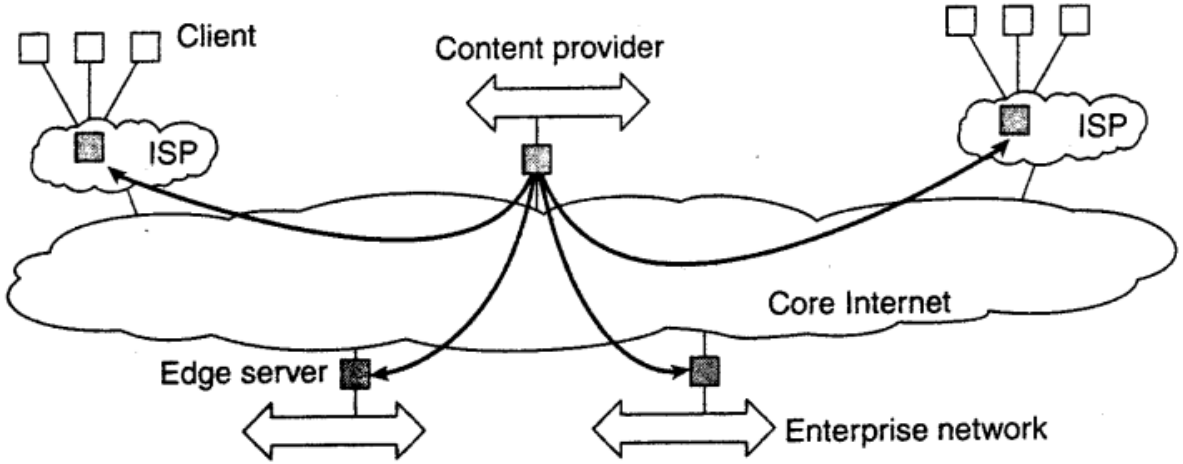
客户端→中央服务

以及

节点/服务↔节点/服务

解释：

- ❑ 登录可能在中心
- ❑ 缓存可以分布式
- ❑ 内容传递可以分发
- ❑ 协调可能仍然处于核心地位



混合架构的例子
Examples of hybrid architecture

示例1：BitTorrent

- ❑ 追踪器或发现服务可以集中管理
- ❑ 实际的文件共享是点对点进行的

为什么选择混合动力？

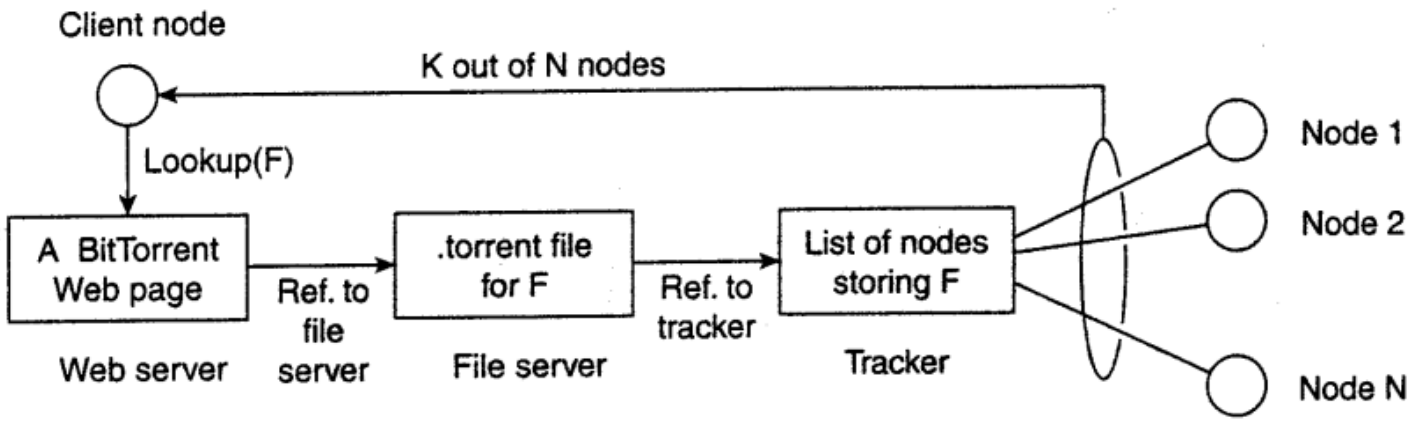
因为它结合了集中协调和去中心化的数据共享。

Example 1: BitTorrent

- ❑ tracker or discovery service may be centralized
- ❑ actual file sharing happens peer-to-peer

Why hybrid?

Because it combines central coordination with decentralized data sharing.



混合架构的例子
Examples of hybrid architecture

Example 2: Large e-commerce platform

- For example, a Taobao-like system:
  - central login/account service
  - distributed product services
  - distributed caches
  - multiple order/payment services

Why hybrid?

Because some functions need central control, while others are spread across many services.

示例2：大型电商平台

- 例如，类似淘宝的系统：
  - 中央登录/账户服务
  - 分布式产品服务
  - 分布式缓存
  - 多项订单/支付服务

为什么选择混合动力?

因为有些功能需要集中控制，而另一些则分散在多个服务中。

混合架构的例子
Examples of hybrid architecture

Example 3: Cloud systems

- ❑ central management dashboard
- ❑ many distributed compute/storage nodes

Why hybrid?

Because users want one controlled platform, but the actual work is distributed.

示例3：云系统

中央管理仪表盘

许多分布式计算/存储节点

为什么选择混合动力?

因为用户想要一个受控的平台，但实际工作是分布式的。

为什么许多真实系统是混合的
Why many real systems are hybrid

- In the real world, very few large systems are purely centralized or purely decentralized.
- Most practical systems mix ideas to get the best balance.

Why?

Because systems need:

- Control
- Scalability
- Resilience
- Manageability
- Hybrid systems try to balance these goals.

- 在现实世界中，很少有大型系统是完全集中或完全去中心化的。
- 大多数实用系统会混合想法以达到最佳平衡。

为什么?

因为系统需要：

- 控制
- 可扩展性
- 韧性
- 可管理性
- 混合系统试图平衡这些目标。

与集中式架构的比较
Comparison with centralized architecture

Architecture	Simple meaning	Easy example	Main strength	Main weakness
Centralized	one main center	campus portal	easy to manage	bottleneck
Decentralized	no single center	P2P file sharing	resilient	hard to coordinate
Hybrid	mix of both	Taobao-like system	balanced design	more complex

Quiz

Distributed System Architecture Q2

Scan with WeChat or long-press

中间件与自我管理
Middleware and Self-Management

中间件与自我管理
Middleware and Self-Management

Learning objectives

- ❑ explain what middleware is in simple terms
- ❑ describe why middleware is important in distributed systems
- ❑ identify common services provided by middleware
- ❑ explain what self-management means
- ❑ understand why large distributed systems need automatic monitoring and adaptation
- ❑ connect these ideas to real systems like WeChat, Taobao, Baidu, and cloud platforms

学习目标

- ❑ 用简单的话解释一下中间件是什么
- ❑ 请描述中间件在分布式系统中为何重要
- ❑ 识别中间件提供的共同服务
- ❑ 解释什么是自我管理
- ❑ 理解大型分布式系统为何需要自动监控和适应
- ❑ 将这些想法与微信、淘宝、百度和云平台等真实系统结合起来

快速回顾
Quick review

we studied how distributed systems can be organized:

- ❑ Centralized
- ❑ Decentralized
- ❑ Hybrid

If many components are spread across different machines, what helps them work together smoothly?

Expected answer:

- ❑ communication software
- ❑ APIs
- ❑ Protocols
- ❑ some “middle” software

That “middle” software is called middleware.

我们研究了分布式系统如何被组织：

- ❑ 集中式
- ❑ 去中心化
- ❑ 混合动力

如果许多组件分布在不同的机器上，是什么帮助它们顺畅协同工作？

预期答案：

- ❑ 通信软件
- ❑ API
- ❑ 协议
- ❑ 一些“中间”软件

这个“中间”软件叫做中间件。

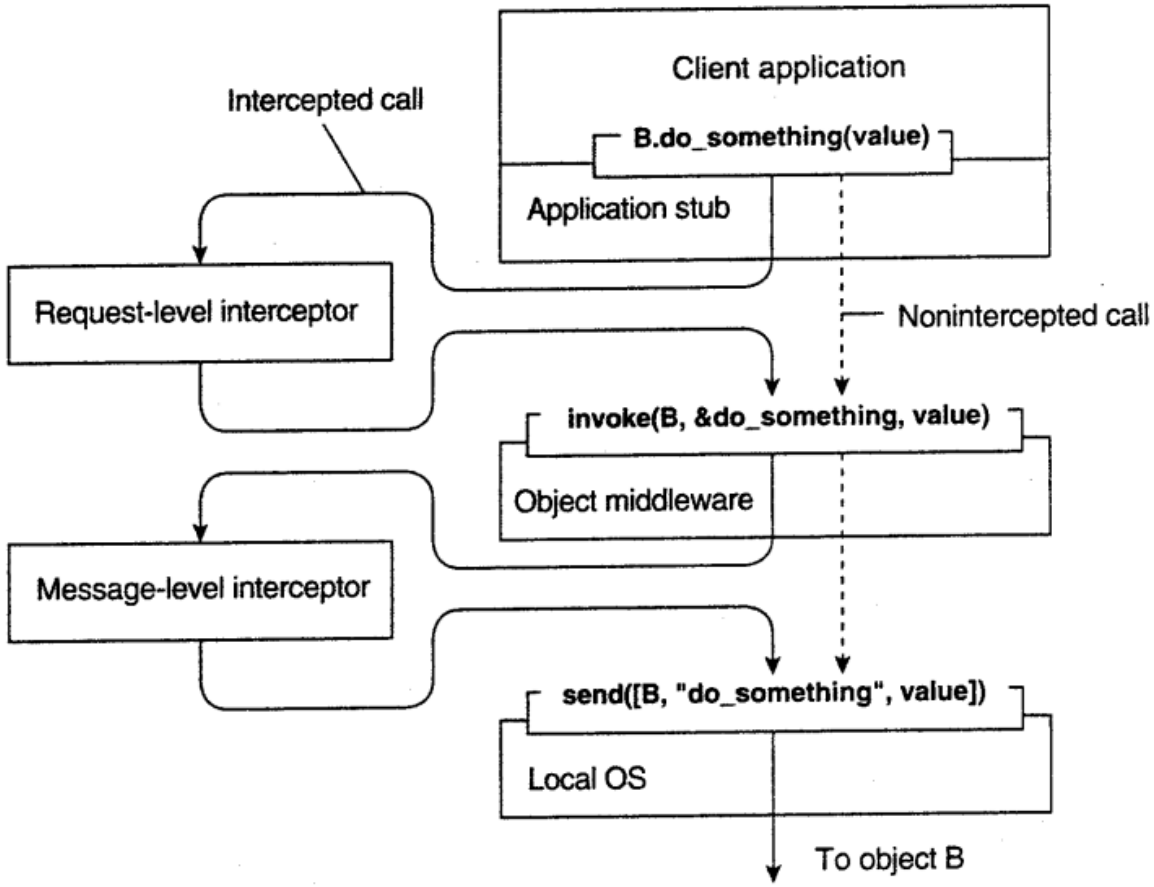
什么是中间件?
What is middleware?

Middleware is software that sits between the application and the underlying distributed system, helping different parts communicate and work together.

Middleware = software in the middle that helps everything connect.
中间件 = 位于中间、帮助系统各部分连接和协同的软件。

Easy analogy
- ❑ Imagine an international meeting.
- ❑ People speak different languages.
- ❑ A translator helps everyone understand each other.
- ❑ Middleware is like that translator.

Another analogy:
If distributed systems are a big company, middleware is like the office system that helps departments send messages, find files, and follow common rules.



为什么需要中间件
Why middleware is needed

Problem without middleware

Without middleware, each developer must handle:

- ❑ low-level communication
- ❑ data formatting
- ❑ naming and locating services
- ❑ error handling
- ❑ security support
- ❑ message delivery

That becomes complex very quickly.

What middleware does?

Middleware reduces this complexity by providing common tools and mechanisms.

没有中间件的问题

没有中间件，每个开发者必须处理：

- ❑ 低层次通信
- ❑ 数据格式化
- ❑ 命名与定位服务
- ❑ 错误处理
- ❑ 安全支持
- ❑ 消息传递

这很快就变得复杂起来。

中间件做什么？

中间件通过提供通用工具和机制来降低这种复杂性。

中间件的核心功能
Core functions of middleware

1. Communication support

Middleware helps components send and receive messages.

Examples

- ❑ request/response
- ❑ remote calls
- ❑ message passing

Middleware helps systems talk to each other.

2. Naming and discovery

Middleware can help a client find:

- ❑ a service
- ❑ a server
- ❑ a resource

Example

A service wants to find the address of another service without hard-coding everything.

Middleware helps systems find each other.

1. 通信支持

中间件帮助组件发送和接收消息。

示例

- ❑ 请求/回复
- ❑ 远程呼叫
- ❑ 消息传递

中间件帮助系统相互通信。

2. 命名与发现

中间件可以帮助客户端找到：

- ❑ 一项服务
- ❑ 一台服务器
- ❑ 一个资源

示例

服务希望找到另一个服务的地址，但不把所有内容硬编码。

中间件帮助系统相互寻找。

中间件的核心功能
Core functions of middleware

3. Data conversion / interoperability

Different systems may use different formats or languages.

- ❑ Middleware helps convert or standardize communication.
- ❑ Middleware helps different systems understand each other.

4. Security support

Middleware may provide:

- ❑ Authentication
- ❑ encryption support
- ❑ access control hooks

Middleware helps protect communication.

3. 数据转换/互操作性

不同系统可能使用不同的格式或语言。

- ❑ 中间件有助于转换或标准化通信。
- ❑ 中间件帮助不同系统相互理解。

4. 安全支持

中间件可能提供:

- ❑ 认证
- ❑ 加密支持
- ❑ 门禁钩

中间件有助于保护通信。

中间件的核心功能
Core functions of middleware

5. Reliability support

Middleware may help with:

- ❑ Retries
- ❑ Acknowledgments
- ❑ Queueing
- ❑ failover support

Middleware helps communication stay dependable.

6. Transparency support

Middleware helps hide distribution details from users and programmers.

Examples:

- ❑ service location hidden
- ❑ communication details hidden
- ❑ some failures hidden

5. 可靠性支持

中间件可能有助于:

- ❑ 重赛
- ❑ 致谢
- ❑ 排队
- ❑ 故障切换支持

中间件有助于保持通信的可靠性。

6. 透明度支持

中间件帮助用户和程序员隐藏分发详情。

示例：

- ❑ 服务位置隐藏
- ❑ 通信细节隐藏
- ❑ 一些失败被隐藏

中间件示例
Middleware examples

Example 1: RPC framework

A program on one machine can call a function on another machine almost as if it were local.

Explanation

- client makes a request
- middleware sends it over network
- server handles it
- result comes back

RPC middleware makes remote calls look easier.

示例1：RPC框架

一台机器上的程序几乎可以像本地程序一样调用另一台机器上的函数。

解释

- 客户提出请求
- 中间件通过网络发送
- 服务器负责处理
- 结果回来了

RPC中间件让远程通话看起来更简单。

中间件示例
Middleware examples

Example 2: Message queue
A sender sends a message to a queue.
The receiver takes it later.

Why useful?
sender and receiver do not need to act at exactly the same time

- ❑ better decoupling
- ❑ helps scaling

Easy systems example
Taobao order system:

- ❑ payment service sends event
- ❑ warehouse service reads event later

示例2：消息队列
发送方向队列发送消息。
接球者之后再拿。

为什么有用?
发送方和接收方不必完全同时行动

- ❑ 更好的解耦
- ❑ 有助于扩展

简单系统示例
淘宝顺序系统:

- ❑ 支付服务发送事件
- ❑ 仓库服务读取事件后期

中间件示例
Middleware examples

Example 3: Application server / service framework

Provides common support for:

- ❑ service registration
- ❑ Configuration
- ❑ Communication
- ❑ Logging
- ❑ Monitoring

Example 4: Service discovery system

When one microservice needs another, middleware helps it locate the correct instance.

示例3：应用服务器/服务框架

提供以下支持:

- ❑ 服务注册
- ❑ 配置
- ❑ 交通
- ❑ 伐木
- ❑ 监测

示例4：服务发现系统

当一个微服务需要另一个时，中间件帮助它找到正确的实例。

现实世界的例子
real-world examples

WeChat

Middleware may help:

- ❑ message routing
- ❑ service communication
- ❑ load handling
- ❑ reliable delivery support

微信

中间件可能有帮助:

- ❑ 消息路由
- ❑ 服务通信
- ❑ 载荷搬运
- ❑ 可靠的配送支持

Taobao

Middleware may help:

- ❑ order service talk to payment service
- ❑ payment service talk to inventory service
- ❑ inventory service talk to shipping service

淘宝

中间件可能有帮助:

- ❑ 订购服务，通话到支付服务
- ❑ 支付服务 与库存服务沟通
- ❑ 库存服务与运输服务沟通

现实世界的例子
real-world examples

Baidu

Middleware may help:

- ❑ search services talk to ranking services
- ❑ indexing services exchange data
- ❑ distributed components communicate reliably

Cloud platforms

Middleware helps:

- ❑ service orchestration
- ❑ resource discovery
- ❑ Monitoring
- ❑ retries and failover

百度

中间件可能有帮助:

- ❑ 搜索服务与排名服务的通信
- ❑ 索引服务交换数据
- ❑ 分布式组件能够可靠地通信

云平台

中间件的帮助:

- ❑ 服务编排
- ❑ 资源发现
- ❑ 监测
- ❑ 重试与故障切换

架构与中间件的比较
architectures versus middleware

Architecture answers:

- how is the system organized?
- who talks to whom?
- where are the components?

Middleware answers:

- how do these components actually cooperate smoothly?
- what common support helps them communicate and function?

Architecture = the structure
Middleware = the helper software that makes the structure work well

Easy analogy

- Architecture is the city map.
- Middleware is the transport and communication system that helps the city function.

建筑学的回答：

- 这个系统是如何组织的？
- 谁和谁说话？
- 组件在哪里？

Middleware的回答：

- 这些组件实际上是如何顺利协作的？
- 什么共同的支持帮助他们沟通和正常运作？

建筑 = 结构
中间件 = 使结构良好运行的辅助软件

简单的比喻

- 建筑就是城市地图。
- 中间件是帮助城市运转的交通和通信系统。

什么是自我管理?
What is self-management?

Self-management means that a distributed system can:

- monitor itself
- detect problems
- adapt automatically
- recover or optimize without needing constant human control

Self-management = the system watches itself and adjusts itself.

自我管理 = 系统自己监控、自己调整。

Why it matters; Modern distributed systems can be:

- very large
- geographically distributed
- always running
- too complex for humans to control manually all the time
- So systems need some level of automation.

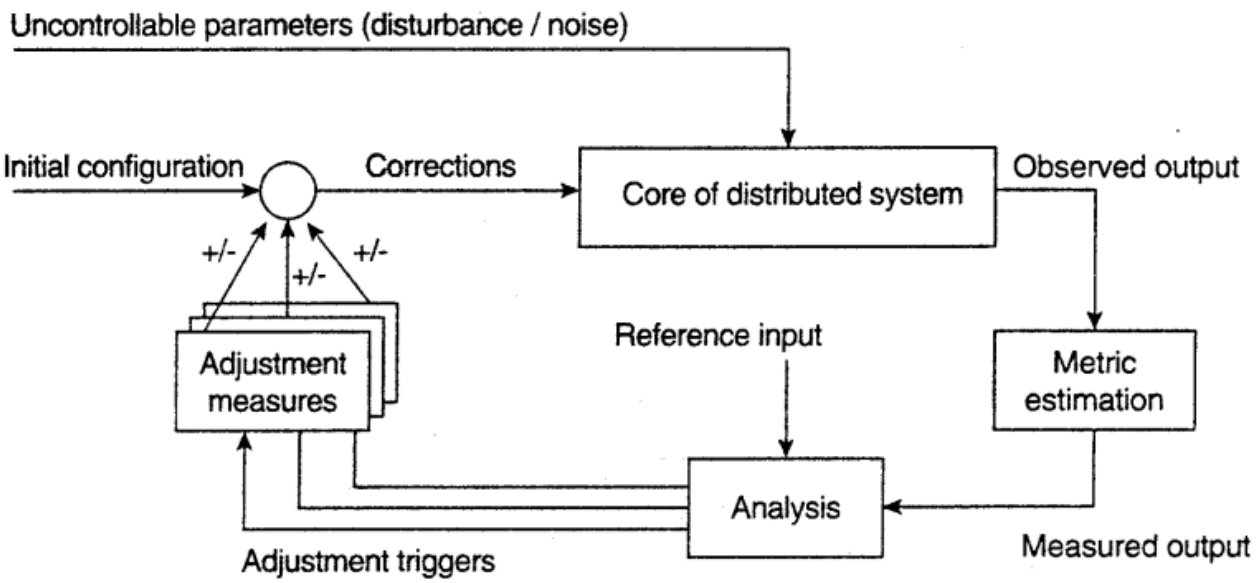


Figure 2-16. The logical organization of a feedback control system.

主要自我管理任务
Main self-management tasks

1. Monitoring

The system checks:

- ❑ CPU usage
- ❑ Memory
- ❑ network traffic
- ❑ error rate
- ❑ response time

First, the system observes itself.

2. Failure detection

The system notices when:

- ❑ a server is down
- ❑ a service stops responding
- ❑ a machine becomes too slow

Then it notices problems.

1. 监测

系统会检查：

- ❑ CPU 使用率
- ❑ 记忆
- ❑ 网络流量
- ❑ 错误率
- ❑ 响应时间

首先，系统观察自己。

2. 故障检测

系统会在以下情况下检测：

- ❑ 服务器宕机了
- ❑ 服务停止响应
- ❑ 机器变得太慢

然后它发现了问题。

主要自我管理任务
Main self-management tasks

3. Recovery
The system may:

- ❑ restart a service
- ❑ reroute traffic
- ❑ switch to another replica
- ❑ isolate a failed node

Then it tries to fix the problem.

4. Load balancing / adaptation
If traffic grows:

- ❑ add more servers
- ❑ move work elsewhere
- ❑ balance requests

Then it adjusts itself to keep working well.

3. 恢复
该系统可能：

- ❑ 重启服务
- ❑ 改道交通
- ❑ 切换到另一个副本
- ❑ 隔离一个失败的节点

然后它试图修复问题。

4. 负载均衡/适应
如果流量增长：

- ❑ 增加更多服务器
- ❑ 把工作转移到别处
- ❑ 平衡请求

然后它会自我调整，继续正常工作。

主要自我管理任务
Main self-management tasks

5. Scaling

If demand increases:

- ❑ scale out
- ❑ allocate more resources

If demand drops:

- ❑ reduce resources

The system grows or shrinks as needed.

5. 规模化

如果需求增加:

- ❑ 规模缩减
- ❑ 分配更多资源

如果需求下降:

- ❑ 减少资源

系统会根据需要增大或缩小。

自我管理的简单类
Easy analogy for self-management

Think of a smart building.

- ❑ sensors check temperature
- ❑ system notices if room is too hot
- ❑ air conditioning turns on automatically

That is self-management.

Now connect with distributed systems:

- ❑ monitoring tools are the sensors
- ❑ controllers make decisions
- ❑ servers and services are adjusted automatically

想象一下智能建筑。

- ❑ 传感器检测温度
- ❑ 系统会检测房间是否过热
- ❑ 空调会自动启动

这就是自我管理。

现在连接分布式系统：

- ❑ 监控工具是传感器
- ❑ 控制者做决策
- ❑ 服务器和服务会自动调整

反馈控制模型
Feedback control model

A basic self-management loop is:

- observe the system
- compare with desired condition
- decide what to do
- apply a correction

Example

- traffic is too high
- response time becomes slow
- system adds more server instances

Observe → decide → act

一个基本的自我管理循环是:

- 观察系统
- 与期望条件进行比较
- 决定接下来该怎么办
- 应用修正

示例

- 交通量太大了
- 响应时间变慢
- 系统增加了更多服务器实例

观察→决定→行动

中间件 + 自我管理
Middleware + self-management together

- Middleware helps distributed components communicate and cooperate.
- Self-management helps the system monitor itself and adapt when conditions change.
- Middleware helps the system work.
- Self-management helps the system keep working well.

- 中间件帮助分布式组件进行通信和协作。
- 自我管理帮助系统自我监控并在条件变化时进行适应。
- 中间件帮助系统正常运行。
- 自我管理有助于系统持续良好运作。

Quiz

Distributed System Architecture Q3

Scan with WeChat or long-press

设计一个简单的客户端-服务器架构，并使用套接字编程实现服务器-客户端通信

Design a Simple Client-Server Architecture and Implement Server-Client Communication Using Socket Programming

什么是套筒?
What is a socket?

A socket is a software endpoint used for communication between two programs over a network.

Socket = communication door.
套接字 = 程序之间通信的门。

A socket lets:

- ❑ one process send data
- ❑ another process receive data

Think of it like:

- ❑ one side opens a door
- ❑ the other side connects to it
- ❑ then they can exchange messages

套接字是一种软件端点，用于网络上两个程序之间的通信。

插槽=通信门。
套接字 = 程序之间通信的门。

一个套筒允许:

- ❑ 一个进程发送数据
- ❑ 另一个进程接收数据

可以这样想:

- ❑ 一侧打开了一扇门
- ❑ 另一端连接到它
- ❑ 然后他们可以交换消息

IP地址和端口
IP address and port

IP address
The IP address tells us which machine to contact.
Example: 127.0.0.1 means the local machine

Port
The port tells us which application or service on that machine to contact.

Example:
one port for web service
another port for chat service

IP = which house
Port = which room

IP地址
IP地址告诉我们该联系哪台机器。
示例：127.0.0.1 表示本地机器

移植
端口告诉我们该机器上应该联系哪个应用或服务。

示例：
一个Web服务端口
又一个聊天服务移植版

IP = 哪个学院
端口 = 哪个房间

基本通信流程
Basic communication flow

Server side

i. create socket
ii. bind socket to IP and port
iii. listen for connections
iv. accept a client connection
v. receive request
vi. process requests
vii. end response
viii. close connection

Client side

i. create socket
ii. connect to server IP and port
iii. send request
iv. receive response
v. close connection

服务器端

i. 创建套接字
ii. 将套接字绑定到IP和端口
iii. 倾听联系
iv. 接受客户端连接
v. 接收请求
vi. 进程请求
vii. 结束反应
viii. 密切联系

客户端

i. 创建套接字
ii. 连接到服务器 IP 和端口
iii. 发送请求
iv. 接收响应
v. 密切联系

[分布式系统：定义]
Basic communication flow practical task

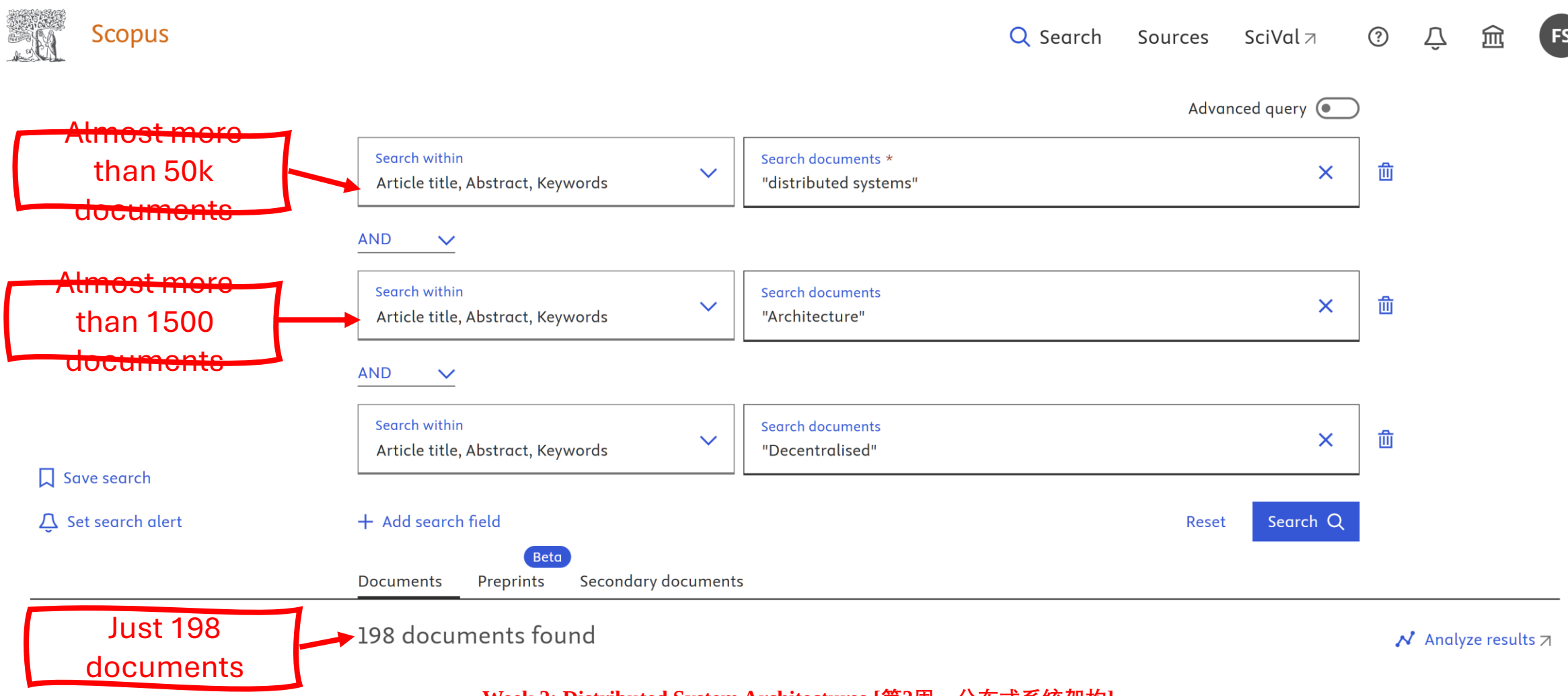
Option 1: Echo client-server
What it does
client sends text
server sends the same text back
Example
Client sends:
Hello server
Server returns:
Hello server

Option 2: Calculator client-server
What it does
Client sends an operation, server calculates result.
Example
Client sends:
ADD 2 3
Server returns:
5

选项1：Echo客户端-服务器
它的作用
客户端发送短信
服务器会把同样的文本反馈回去
示例
客户发送：
你好，服务器
服务器回归：
你好，服务器

选项2：计算器客户端-服务器
它的作用
客户端发送操作，服务器计算结果。
示例
客户发送：
ADD 2 3
服务器返回：5

调查论文示例/想法
Example/Idea for Survey Paper



调查论文示例/想法
Example/Idea for Survey Paper

Possible Paper Titles:

- ❑ Decentralised Architectures for Distributed Systems: A Systematic Survey of Designs, Trade-offs, and Future Directions
- ❑ Decentralised Architectures for Distributed Systems: A Systematic Literature Review
- ❑ A PRISMA-Based Systematic Review of Decentralised Architectures in Distributed Systems
- ❑ Architectural Taxonomies for Decentralised Systems: A Systematic Literature Review

调查论文示例/想法
Example/Idea for Survey Paper

Option(s) we have for the documents analysis

1. Use Bibliometrix Tools to perform analyses over the data extracted
   i. VosViewer
   ii. RStudio (bibliometrix/bibshiny)
   iii. Can also have implementation of bibliometrics analyses in Python or other languages.
   iv. And many others…

2. Perform semantic/thematic analyses in Python.

3. A PRISMA-Based Systematic Review and categorization of the existing works.

调查论文示例/想法
Example/Idea for Survey Paper

Go to file/function | Addins | Project: (None)

Console | Terminal | Background Jobs

R 4.4.2 · ~/

```
lowing reference:

Aria, M. & Cuccurullo, C. (2017) bibliometrix: An R-tool for comprehensive
science mapping analysis,
                        Journal of Informetrics, 11(4), pp 959-975, Elsevie
r.

Failure to properly cite the software is considered a violation of the lice
nse.

For information and bug reports:
                        - Take a look at https://www.bibliometrix.org
                        - Send an email to info@bibliometrix.org
                        - Write a post on https://github.com/massimoaria/bi
bliometrix/issues

Help us to keep Bibliometrix and Biblioshiny free to download and use by co
ntributing with a small donation to support our research team (https://bibl
iometrix.org/donate.html)


To start with the Biblioshiny app, please digit:
biblioshiny()


Warning message:
package 'bibliometrix' was built under R version 4.4.3

> biblioshiny()
```

Environment | History | Connections | Tutorial

Import Dataset | 400 MiB | List

R | Global Environment

Environment is empty

Files | Plots | Packages | **Help** | Viewer | Presentation

R: Run Shiny Application | Find in Topic

runApp {shiny} R Documentation

Run Shiny Application

Description

Runs a Shiny application. This function normally does not return; interrupt R to stop the application (usually by pressing Ctrl+C or Esc).

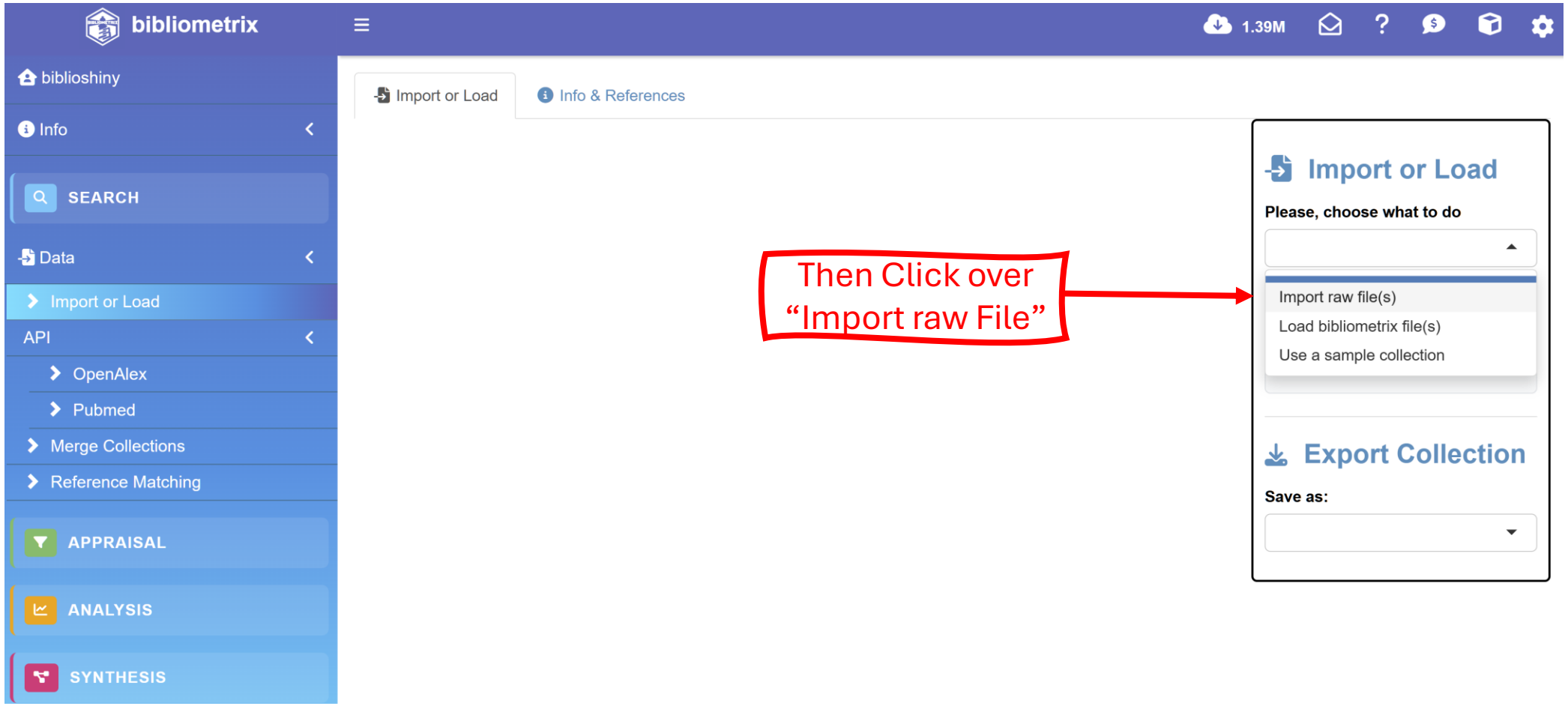
Usage

```
runApp(
  appDir = getwd(),
  port = getOption("shiny.port"),
  launch.browser = getOption("shiny.launch.browser", interacti
```

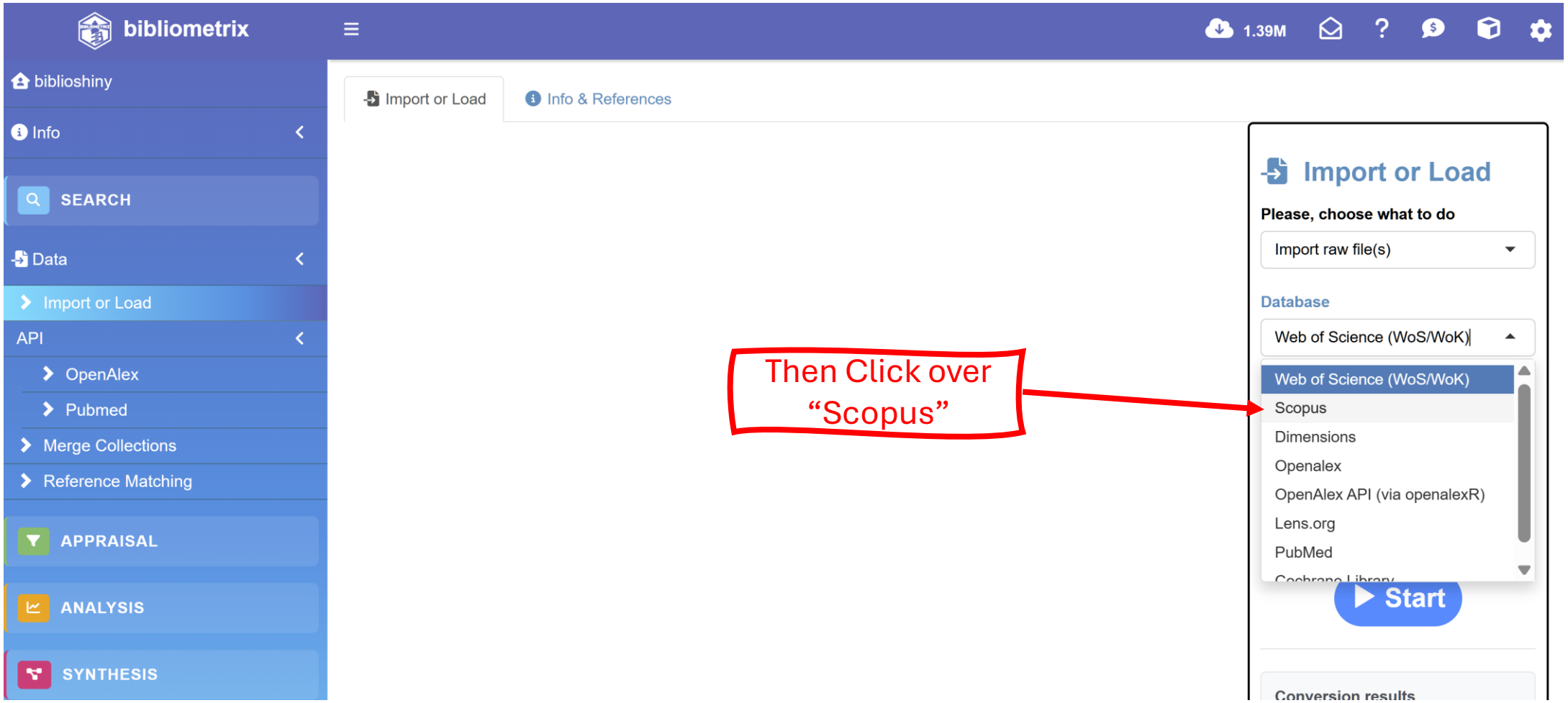
调查论文示例/想法
Example/Idea for Survey Paper

调查论文示例/想法
Example/Idea for Survey Paper

调查论文示例/想法
Example/Idea for Survey Paper

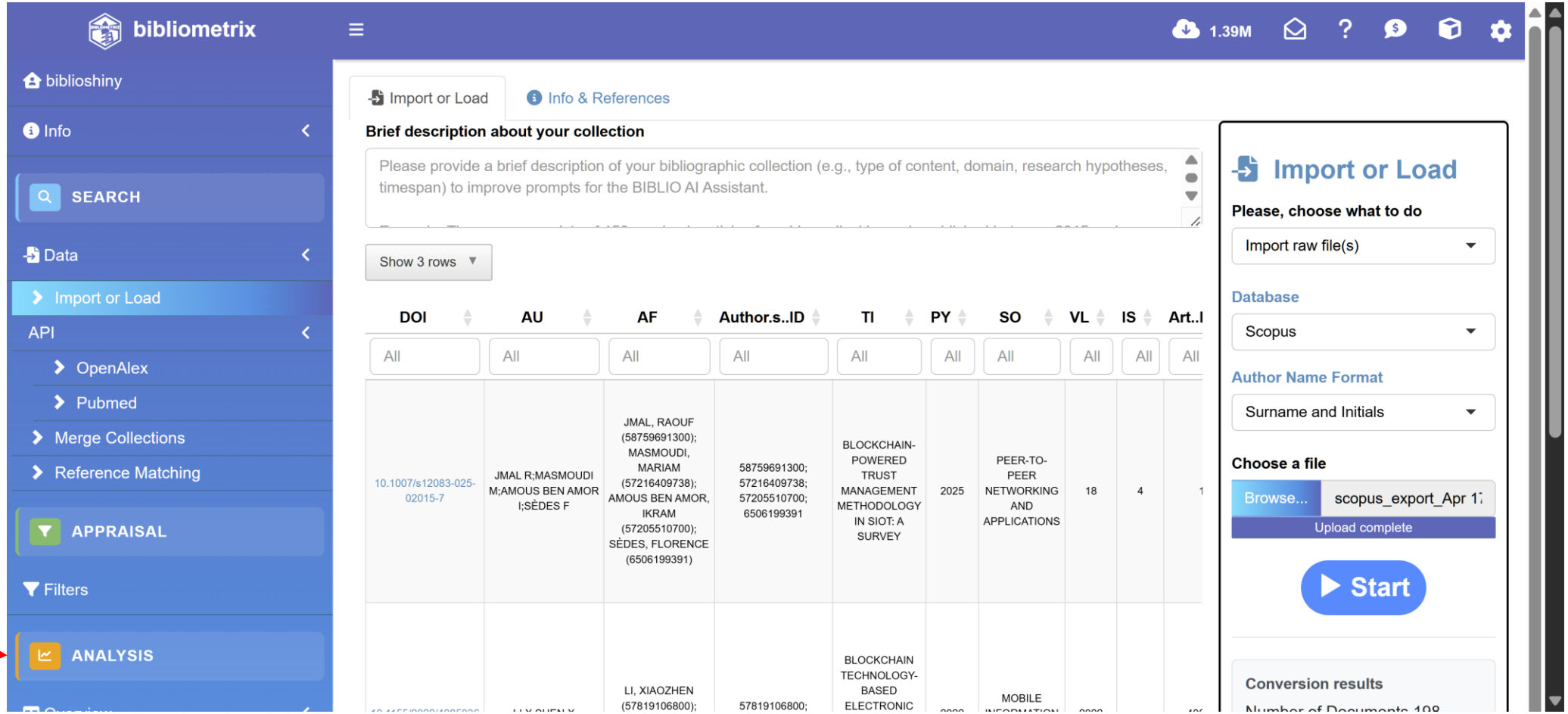


调查论文示例/想法
Example/Idea for Survey Paper



调查论文示例/想法
Example/Idea for Survey Paper

Once you the data uploaded/imported to the bibliometrix



另一个调查论文的示例/想法
Another Example/Idea for Survey Paper

Scopus

Search Sources SciVal FS

Advanced query

Search within
Article title, Abstract, Keywords

Search documents *
"distributed systems"

AND

Search within
Article title, Abstract, Keywords

Search documents
"Architecture"

AND

Search within
Article title, Abstract, Keywords

Search documents
"Middleware"

Save search

Set search alert

Add search field

Reset

Search

Beta

Documents Preprints Secondary documents

43 documents found

Analyze results

Final takeaway for Week 2 第二周总结

Distributed System Architectures:

Architecture = How the system is organized

Three main architecture types

Middleware = Helper in the middle

Self-Management = System manages itself

Client server is the foundation